

Презентация по лабораторной работе № 7

Основы информационной безопасности

Луангсуваннавонг Сайпхачан

Содержание I

1. Информация
2. Выполнение лабораторной работы
3. Выводы

Раздел 1

1. Информация

1.1 Докладчик

- Луангсуваннавонг Сайпхачан

1.1 Докладчик

- Луангсуваннавонг Сайпхачан
- студент группы НКАбд-01-24

1.1 Докладчик

- Луангсуваннавонг Сайпхачан
- студент группы НКАбд-01-24
- Российский университет дружбы народов им. П. Лумумбы

1.1 Докладчик

- Луангсуваннавонг Сайпхачан
- студент группы НКАбд-01-24
- Российский университет дружбы народов им. П. Лумумбы
- <https://github.com/sayprachanh-lsvnv>

1.2 Цель работы

Освоить на практике применение режима однократного гаммирования

Раздел 2

2. Выполнение лабораторной работы

2.1 Выполнение лабораторной работы

Я буду выполнять лабораторную работу на языке программирования Python. Для этой лабораторной работы требуется разработать программу, позволяющую шифровать и дешифровать данные в режиме однократного использования (гаммирования). Сначала я создаю функцию для генерации случайного ключа с использованием системного генератора случайных чисел.

```
1 import os
2
3 def generate_key(text):
4     return os.urandom(len(text))
5
```

Рисунок 1: Функция генерирования ключа

2.2 Выполнение лабораторной работы

Согласно примеру в лабораторной работе, необходимо, чтобы все данные отображались в шестнадцатеричном формате, поэтому я создаю функцию, которая преобразует открытый текст в шестнадцатеричный формат.

```
5  
6 def txt_to_hex(text):  
7     return ' '.join(f'{c:02X}' for c in text)  
8
```

Рисунок 2: Функция преобразования текстового формата

2.3 Выполнение лабораторной работы

Создание функции, которая выполняет одновременное шифрование и дешифрование с использованием операции «исключающее ИЛИ» (XOR). Поскольку сама операция обратима, я создаю только одну функцию как для шифрования, так и для дешифрования текста.

```
8  
9 def en_de_crypt(text, key):  
10     return bytes([b ^ k for b,k in zip(text, key)])  
11
```

Рисунок 3: Функция шифрования и дешифрования

2.4 Выполнение лабораторной работы

Необходимо определить ключ, с помощью которого шифротекст можно преобразовать во фрагмент текста, представляющий один из возможных способов прочтения открытого текста. Поэтому я создаю функцию, которая будет находить возможные ключи для различных позиций фрагмента в шифротексте.

```
11
12 def find_possible_key(fragment, cipher):
13     key = []
14     for i in range(len(cipher) - len(fragment) + 1):
15         part = cipher[i:i+len(fragment)]
16         key_part = en_de_crypt(part, fragment)
17         key.append(key_part)
18     return key
19
```

Рисунок 4: Функция поиска возможного ключа

2.5 Выполнение лабораторной работы

Я запускаю функцию и проверяю результат. Шифрование и дешифрование работают корректно, выдавая результат текста до и после шифрования, а также в шестнадцатеричном формате.

```
original text:
С Новым Годом, друзья!
key:
A5 29 DB 15 87 C0 1D E8 F4 78 B6 65 FE DE C8 A5 4D 5D 6F 4A C9 AA DA B9 54 85 76 DD 35 10 4A 02 65 0F 4D 2D D2 D3
BA
encrypt (hex):
75 88 FB C5 1A 10 A3 38 46 A9 3D B5 42 FE 18 36 9D E3 BF FE 19 14 0A 05 78 A5 A6 69 E4 90 9B 81 B5 B8 9C A1 03 5C
9B
decrypt (hex):
D0 A1 20 D0 9D D0 BE D0 B2 D1 8B D0 BC 20 D0 93 D0 BE D0 B4 D0 BE D0 BC 2C 20 D0 B4 D1 80 D1 83 D0 B7 D1 8C D1 8F
21
decrypt (text)
С Новым Годом, друзья!
```

Рисунок 5: Шифрование и дешифрование

2.6 Выполнение лабораторной работы

А также поиск возможных ключей, которые можно использовать для дешифровки только части текста (фрагмента).

possible key fragment:

A5	29	DB	15	87	C0	1D	E8	F4	78	B6	65	FE	DE
58	5A	E5	CA	8D	73	86	96	1B	EC	3E	92	42	38
2B	64	3A	C0	3E	E8	F8	79	8F	64	C9	2E	A4	16
15	BB	30	73	A5	96	17	ED	07	93	75	C8	8A	BD
CA	B1	83	E8	DB	79	83	65	F0	2F	93	E6	21	C3

recover fragment: С Новым

Рисунок 6: Возможные ключи

Раздел 3

3. Выводы

3. Выводы

в этой лабораторной работе, я освоил на практике применение режима однократного гаммирования